Day 1 — Production Server Audit

Welcome to your first day. No warmup. No handholding.

Scenario:
You've just been handed SSH access to a production Linux server you've never seen before. The previous admin is gone. No documentation. Server is live.

Question:
What are the FIRST 5 things you check — and why?
Don't give me a list of commands. Give me what, why, and what you're looking for.
Think like the server could be compromised, misconfigured, or about to fall over.
I'm waiting. Take your time — but be precise.

My Evaluation

Your Check	Reality Check
`ping`	Ping what? Localhost? Gateway? External? Be specific.
services	`systemctl list-units --type=service --state=running` — but WHY? Looking for what exactly?
users	`who` / `w` / `last` — good. But also `/etc/passwd`, logged-in vs ALL users
uptime	✅ Solid. `uptime` or `cat /proc/uptime` — tells you if recent reboot (suspicious?)
`df -h`	✅ Good. But also `lsblk` — mounts, disk layout
memory	`free -h` — correct tool
`who`	✅ But `w` gives MORE — what they're running

What a Senior Actually Runs First

bash

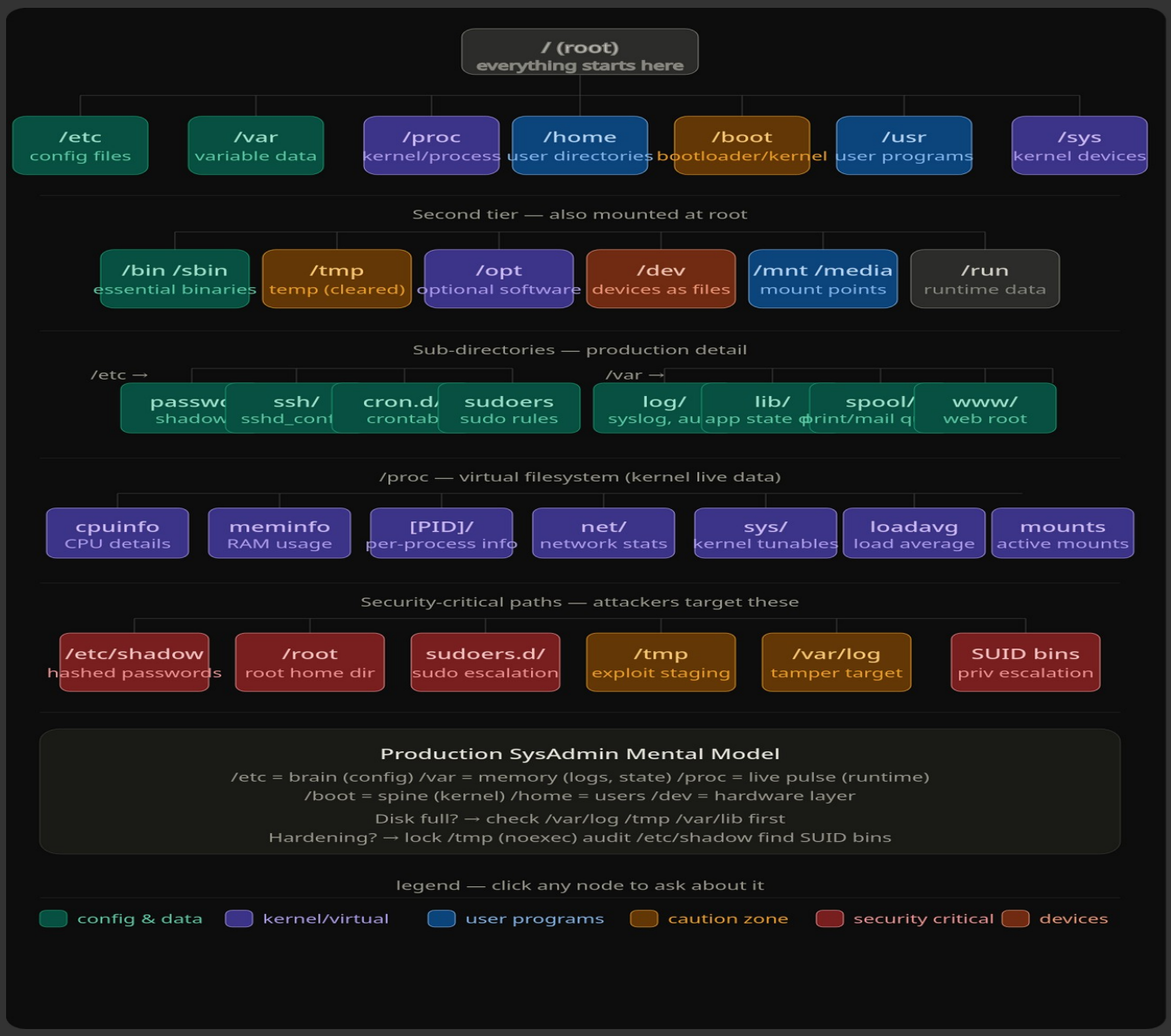
```bash
# 1. Who am I, where am I
whoami && hostname && uname -r

# 2. How long up, load average
uptime

# 3. Who else is logged in RIGHT NOW
w

# 4. Last logins (spot anomalies)
last | head -20


# 5. Failed login attempts
lastb | head -20       # or: journalctl | grep "Failed"

# 6. Running services
systemctl list-units --type=service --state=running

# 7. Open ports (what's exposed)
ss -tlnp

# 8. Disk
df -h

# 9. Memory + swap
free -h

# 10. Recent critical logs
journalctl -p 3 -xb
```

The one thing most juniors **miss** → `lastb` and open ports. That's where attacks live.

Day 1 — Linux filesystem deep dive + system audit

Objective:

Understand the Linux filesystem hierarchy at a production level and build a real system audit script that any SysAdmin would actually use on a new server.



Here's the filesystem map you'll be navigating today — every directory, what lives there, and why it matters in production:

Unable to reach visualize

Click any directory in the diagram to ask questions about it. Now let's get your hands dirty.

Step-by-step commands + explanations

Phase 1 — read the machine

bash

```
# Who are you and what machine is this?
whoami && hostname && uname -a
```

`uname -a` prints kernel version, architecture, and build date. In production you use this to check if a kernel patch is applied.

bash

```
# How long has this server been running? (uptime = trust level)
uptime
```

A server with 400+ days uptime might mean "nobody dares reboot it" — a red flag in production.

bash

```
# What OS version exactly?
cat /etc/os-release
lsb_release -a
```

`/etc/os-release` is the authoritative file. `lsb_release` is the human-friendly wrapper. Both matter for scripting.

bash

```
# What's the hardware underneath?
lscpu
free -h
lsblk
```

`lscpu` → CPU cores, threads, architecture. `free -h` → RAM + swap. `lsblk` → disks, partitions, mount points. These three commands are the first thing you run on any new server.

Phase 2 — filesystem navigation like a senior

bash

```
# Disk usage — where is space being eaten?
df -h
df -hT   # -T also shows filesystem TYPE (ext4, xfs, tmpfs)
```

`df -h` shows disk space per mounted partition. `-T` adds filesystem type. `tmpfs` entries (like `/proc`, `/dev/shm`) are RAM-backed — don't panic when they show up.

bash

```bash
# Find what's eating your disk in /var (biggest offender on real servers)
du -sh /var/*
du -sh /var/log/*
```

`du -sh` = disk usage, summarized, human-readable. This is how you find a 50GB log file that took down a production server at 3am.

bash

```bash
# Navigate and inspect like a pro
ls -lah /etc/        # -a shows hidden files, -h human sizes
ls -lah /var/log/
stat /etc/passwd      # Full metadata: permissions, owner, timestamps, inode
```

`stat` gives you the inode number, which is critical when debugging hard links or "file deleted but disk not freed" issues.

bash

```bash
# Find files modified in the last 24 hours (incident response skill)
find /etc -mtime -1 -type f
find /var/log -mtime -1 -name "*.log"
```

`-mtime -1` = modified less than 1 day ago. In incident response, this tells you what changed before the outage.

bash

```bash
# Check inodes (often overlooked — can fill up independently of disk space)
df -i
```

If inode usage hits 100%, you cannot create new files even if disk space is free. This breaks mail servers, web apps, cron jobs silently.

Mini-project — production server audit script

Project name: `server-audit.sh`

Why: The first thing every SysAdmin does on a new server or after an incident — get a full picture fast. Instead of running 15 commands manually, this script does it in one shot.

bash

```bash
# Create the script
nano ~/server-audit.sh
```

Paste this — every line is explained inline:

bash

```bash
#!/bin/bash
# ==============================================================
# server-audit.sh — Production Server Audit Script
# Author: Abdul Basit | Date: $(date)
# Run this on any new Ubuntu server to baseline its state
# ==============================================================

# Report File Save Path
REPORT_FILE="/tmp/audit-$(hostname)-$(date +%Y%m%d).txt"

echo "============================================" | tee $REPORT_FILE
echo "  SERVER AUDIT REPORT" | tee -a $REPORT_FILE
echo "  Generated: $(date)" | tee -a $REPORT_FILE
echo "============================================" | tee -a $REPORT_FILE

# System Identity
echo -e "\n[1] SYSTEM IDENTITY" | tee -a $REPORT_FILE
echo "Hostname : $(hostname)" | tee -a $REPORT_FILE
echo "OS       : $(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '")" | tee -a $REPORT_FILE
echo "Kernel   : $(uname -r)" | tee -a $REPORT_FILE
echo "Uptime   : $(uptime -p)" | tee -a $REPORT_FILE
echo "User     : $(whoami)" | tee -a $REPORT_FILE

# Hardware Resources
echo -e "\n[2] HARDWARE RESOURCES" | tee -a $REPORT_FILE
echo "CPU      : $(lscpu | grep 'Model name' | cut -d: -f2 | xargs)" | tee -a $REPORT_FILE
echo "Cores    : $(nproc)" | tee -a $REPORT_FILE
echo "RAM      : $(free -h | grep Mem | awk '{print $2}')" | tee -a $REPORT_FILE
echo "RAM Used : $(free -h | grep Mem | awk '{print $3}')" | tee -a $REPORT_FILE

# Disk Usage
echo -e "\n[3] DISK USAGE" | tee -a $REPORT_FILE
df -hT | tee -a $REPORT_FILE

# Top 5 Space Consumers in /var
echo -e "\n[4] TOP 5 SPACE CONSUMERS in /var" | tee -a $REPORT_FILE
du -sh /var/* 2>/dev/null | sort -rh | head -5 | tee -a $REPORT_FILE

# Inode Usage
echo -e "\n[5] INODE USAGE" | tee -a $REPORT_FILE
df -i | tee -a $REPORT_FILE

# Network Interfaces
echo -e "\n[6] NETWORK INTERFACES" | tee -a $REPORT_FILE
ip addr show | grep -E "^[0-9]+:|inet " | tee -a $REPORT_FILE

# Listening Ports
echo -e "\n[7] LISTENING PORTS (what services are exposed?)" | tee -a $REPORT_FILE
ss -tlnp | tee -a $REPORT_FILE

# Logged-In Users
echo -e "\n[8] LOGGED-IN USERS" | tee -a $REPORT_FILE
who | tee -a $REPORT_FILE
```

```bash
# Last  10 Logins
echo -e "\n[9] LAST 10 LOGINS" | tee -a $REPORT_FILE
last -n 10 | tee -a $REPORT_FILE

# Failed Login Attempts
echo -e "\n[10] FAILED LOGIN ATTEMPTS" | tee -a $REPORT_FILE
grep "Failed password" /var/log/auth.log 2>/dev/null | tail -5 | tee -a $REPORT_FILE

# Running Services
echo -e "\n[11] RUNNING SERVICES" | tee -a $REPORT_FILE
systemctl list-units --type=service --state=running | tee -a $REPORT_FILE

# Files Modified In Last 24H in /etc
echo -e "\n[12] FILES MODIFIED IN LAST 24H in /etc" | tee -a $REPORT_FILE
find /etc -mtime -1 -type f 2>/dev/null | tee -a $REPORT_FILE

# Report File Save Path
echo -e "\n==========================================" | tee -a $REPORT_FILE
echo "  Report saved to: $REPORT_FILE" | tee -a $REPORT_FILE
echo "==========================================" | tee -a $REPORT_FILE
```

Now run it:

bash

```bash
# Make executable and run
chmod +x ~/server-audit.sh
sudo ./server-audit.sh

# View the saved report
cat /tmp/audit-$(hostname)-$(date +%Y%m%d).txt
```

Why `tee`? It prints to terminal AND writes to file simultaneously. Senior move — you always save audit output.

Why `2>/dev/null`? Suppresses permission errors on files you can't read. Clean output on production.

Why `sort -rh | head -5`? `-r` reverses, `-h` compares human sizes (GB > MB > KB). You get the biggest culprits first.

Day 2 preview

Tomorrow: **User & Group Management** — create users, assign groups, set sudo privileges, and lock down a service account exactly how it's done in a real company with multiple admins on one server.